

TDK-dolgozat

KCMamba: Kalman-Guided Selective State-Space Model for Robust Time Series Forecasting

András Kovács

Computer Science BSc, Year 3

Eötvös Loránd University
Faculty of Informatics

Budapest, May 2026



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

KCMamba: Kalman-Guided Selective State-Space Model for Robust Time Series Forecasting

Supervisor:

Kristian Fenech Arian

lecturer

Author:

András Kovács

Computer Science BSc, Year 3

Budapest, May 2026

Abstract

This thesis presents **KCMamba** (Kalman Core-Attention Mamba), a hybrid neural architecture that integrates a differentiable Kalman filtering mechanism into the Mamba selective state-space model (SSM) for noise-robust time series forecasting. The proposed architecture employs learned Q_{net} , R_{net} , and K_{net} subnetworks that adaptively regulate the ratio of process and measurement noise, enabling the model to behave as a passive predictor on clean data and as an active filter under noisy conditions.

We conduct empirical evaluation on three public LTSF benchmarks (Exchange-Rate, ETTh1, ETTh2) under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$) and low data-availability regimes (stride $\in \{1, 16\}$).

In the Exchange-Rate benchmark ($s = 16$, 3-seed experiment), KCMamba’s average MSE degradation from $\sigma=0$ to $\sigma=5$ is +1528%, compared to +28234% for Fair Mamba, an $18.5\times$ stability gap; in the hardest configuration ($s=16$, $\sigma=5$) KCMamba achieves MSE 1.41 versus 22.72 for Fair Mamba. The ablation study identifies the $Q_{\text{net}}/R_{\text{net}}$ subnetworks as the primary source of robustness: removing the learned K -gain increases degradation by $2.4\times$.

The thesis also describes the complete production software system: an asynchronous, event-driven trading engine built in a hexagonal architecture, a web-based dashboard, a backtesting framework, and a live Binance trading adapter.

Contents

Abstract	1
1 Introduction	5
1.1 Motivation	5
1.2 Research Questions	5
1.3 Contributions	6
1.4 Thesis Structure	6
2 Related Work	7
2.1 The Kalman Filter and Its Extensions	7
2.1.1 Learned Kalman Filters	8
2.2 State Space Models	9
2.2.1 S4	9
2.2.2 Mamba	9
2.2.3 Fair Mamba	10
2.3 Long-Term Time Series Forecasting	10
2.3.1 Problem Formulation	10
2.3.2 Reference Models	11
2.3.3 Datasets	12
2.4 Noise Robustness	12
3 The KCMamba Architecture	13
3.1 Overview	13
3.2 KCMamba Block	15
3.2.1 Input Projection and 1D Convolution	15
3.2.2 Kalman Parameter Networks	15
3.2.3 Kalman Gain Computation	16
3.2.4 State Update with Parallel Scan	17

3.2.5	Output Projection and Residual Connection	18
3.3	KCMamba Stack	18
3.4	Parameter Complexity	21
3.5	Comparison with Standard Mamba	21
3.6	System Architecture	21
3.7	Module and Class Structure	22
4	Experiments and Evaluation	24
4.1	Experimental Setup	24
4.1.1	Hardware and Software	24
4.1.2	Datasets	24
4.1.3	Noise Injection	24
4.1.4	Model Configuration and Training Hyperparameters	25
4.2	Results	25
4.2.1	Exchange-Rate Dataset	25
4.2.2	ETTh1 Dataset	26
4.2.3	ETTh2 Dataset	26
4.2.4	Win Statistics Summary	27
4.3	Kalman Gain Dynamics	29
4.4	Discussion	31
4.5	Long-Sequence Analysis	32
4.6	Stride $\times$ Long-Sequence Interaction	33
4.7	Ablation Study and Statistical Significance	35
5	Conclusion	37
5.1	Main Findings	37
5.2	Limitations	38
5.3	Future Work	38
	Acknowledgements	40
	Statement of Own Contributions	41
A	Use of Artificial Intelligence Tools	42
	Bibliography	42

List of Figures	45
------------------------	-----------

Chapter 1

Introduction

Time series forecasting is one of the most studied areas of applied machine learning, partly because real-world data is almost always noisy. Sensor dropouts, aggregation artefacts, and market microstructure noise all degrade the signal-to-noise ratio, and models frequently struggle with this. In the long-term forecasting (LTSF) setting the problem is harder still, since a lookback window of length L must support a prediction of the next H steps, where H can exceed L [1].

1.1 Motivation

Transformer-based models [1, 2] and the Mamba SSM [3, 4] address data-quality issues through implicit generalisation, relying on the model to learn around the noise. This works on clean data, but there is no guarantee that the robustness transfers to noisy regimes.

The Kalman filter [5] takes a different route: it builds an *explicit* model of process noise (Q) and measurement noise (R), and derives a gain K from their ratio to decide how much to trust the current measurement versus the prior state. The **KCMamba** architecture presented in this thesis is built on this principle. That inductive bias, if embedded in a neural network, may help in noisy conditions.

1.2 Research Questions

Three questions are investigated: (1) is KCMamba competitive on clean data against standard LTSF models; (2) how much MSE degradation do models suffer

under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$); and (3) which architecture generalises better under scarce training data ($\text{stride} \in \{1, 16\}$).

1.3 Contributions

The thesis makes five concrete contributions. The **KCMamba** architecture embeds learned Q_{net} , R_{net} , K_{net} subnetworks inside a differentiable Kalman filter, with a parallel prefix-scan GPU implementation for the recursion. Measurements show that the learned K genuinely adapts to noise: $K \approx 0.71$ at $\sigma = 0$ (passive pass-through) and $K \approx 0.07$ at $\sigma = 5$ (strong filtering).

Benchmark results on three datasets (Exchange-Rate, ETTh1, ETTh2) show that Fair Mamba degrades by 28 234% under noise, whereas KCMamba degrades by only 1 528% ($18.5\times$ smaller degradation). The long-sequence analysis ($T \in \{96, 192, 336, 512\}$, ETTm1 and Exchange) confirms KCMamba’s advantage at longer contexts: at $T=336$, $\sigma=1$ it achieves 31.7% lower MSE than LSTM (0.502 vs 0.735); at $T=512$, $\sigma=3$, KCMamba scores 0.773 while Fair Mamba scores 2.121. On ETTm1 at $\sigma=3$ KCMamba is the only model whose MSE monotonically decreases with sequence length ($0.673 \rightarrow 0.466$, -31% , $T: 96 \rightarrow 512$). In the stride $\times$ sequence-length sweep ($s \in \{1, 16\}$, $T \in \{192, 336, 512\}$), at $s=16$, $\sigma \geq 3$, Fair Mamba’s MSE rises to 8.1–8.5 while KCMamba stays at 1.1–1.3. All experiments are accompanied by an open-source Python/PyTorch/CUDA implementation integrated into an asynchronous live-trading system.

1.4 Thesis Structure

Chapter 2 reviews the Kalman filter, SSMs, and the LTSF literature. Chapter 3 describes the KCMamba architecture and its mathematical underpinnings. Chapter 4 presents the experimental setup and results. Chapter ?? covers the implemented software system. Chapter 5 draws conclusions and discusses future work.

Chapter 2

Related Work

2.1 The Kalman Filter and Its Extensions

The Kalman filter [5] is the minimum-variance, unbiased, linear estimator for systems whose state $x_t \in \mathbb{R}^n$ evolves according to a linear dynamical model perturbed by Gaussian noise:

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad (2.1)$$

$$y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R), \quad (2.2)$$

where F is the state-transition matrix, H the observation matrix, $Q \in \mathbb{R}^{n \times n}$ the process-noise covariance, and $R \in \mathbb{R}^{m \times m}$ the measurement-noise covariance. The filter operates in two alternating half-steps. In the prediction step (time update), before the measurement y_t arrives, the filter propagates the state estimate and the error covariance forward in time:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \quad (2.3)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^\top + Q, \quad (2.4)$$

where $P_{t|t-1} \in \mathbb{R}^{n \times n}$ is the predicted error covariance. A large $P_{t|t-1}$ indicates high prediction uncertainty; a large Q means the system state changes rapidly and unpredictably.

In the correction step (measurement update), once the observation y_t arrives, the filter corrects its estimate by the *innovation* $e_t = y_t - H\hat{x}_{t|t-1}$, weighted by the

Kalman gain K_t :

$$K_t = P_{t|t-1} H^\top (H P_{t|t-1} H^\top + R)^{-1}, \quad (2.5)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t, \quad (2.6)$$

$$P_{t|t} = (I - K_t H) P_{t|t-1}. \quad (2.7)$$

Equation (2.5) encodes the fundamental noise-balance intuition. When the measurement noise R is large relative to the predicted covariance $H P_{t|t-1} H^\top$, the denominator dominates and $K_t \rightarrow 0$: the filter distrusts the observation and relies on its prior state. When $R \rightarrow 0$, $K_t \rightarrow H^{-1}$ and the observation nearly overwrites the state entirely.

For the scalar case ($F = H = 1$), the gain simplifies to

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}. \quad (2.8)$$

Under steady-state conditions the error covariance $P_{t|t-1}$ converges to a fixed value $\bar{P}$, and the steady-state gain depends only on the ratio Q/R : a high process noise relative to measurement noise drives K towards 1 (trust the measurement), while a low Q/R ratio drives K towards 0 (trust the model). This $Q/(Q + R)$ ratio is the structural basis for the KCMamba gain computation (Section 3.2.3).

2.1.1 Learned Kalman Filters

In the neural Kalman literature, the matrices F, H, Q, R are parameterised by neural networks [6, 7]. The *Deep Kalman Filter* [6] allows non-linear transitions, and the *Kalman Variational Autoencoder* [7] learns disentangled representations of latent states. KCMamba builds on these ideas but operates as a sequence forecaster rather than as an autoencoder.

2.2 State Space Models

2.2.1 S4

S4 [3] reformulates a sequence model as a parameterised ordinary differential equation (ODE) over a latent state $h(t) \in \mathbb{R}^N$:

$$h'(t) = Ah(t) + Bu(t), \quad (2.9)$$

$$y(t) = Ch(t). \quad (2.10)$$

Here $u(t)$ is the scalar input, $A \in \mathbb{R}^{N \times N}$ is the state matrix, and $B, C \in \mathbb{R}^N$ are the input and output projections. The critical insight of Gu et al. is that A can be initialised as the *HiPPO* (High-order Polynomial Projection Operator) matrix, which is designed to compress the history of u into the current state $h(t)$ with optimal polynomial approximation. This initialisation avoids the vanishing-gradient problem that plagues vanilla RNNs over long sequences, since the HiPPO structure preserves information from the distant past without exponential decay.

To apply the model to discrete sequences the continuous system is discretised with step size Δ using the zero-order hold (ZOH) rule:

$$h_t = \bar{A}h_{t-1} + \bar{B}u_t, \quad (2.11)$$

$$y_t = Ch_t, \quad (2.12)$$

where $\bar{A} = e^{\Delta A}$ and $\bar{B} = (e^{\Delta A} - I)A^{-1}B$. Because $\bar{A}$ is diagonal in the frequency domain the entire convolution $y = \bar{C} * u$ can be computed as a single element-wise product after a Fast Fourier Transform (FFT), reducing sequence-level complexity to $O(T \log T)$.

A fundamental limitation of S4 is that A , B , C , and Δ are *input-independent*: the same learned parameters are applied at every time step regardless of the input content. The model therefore cannot adapt its memory or forgetting behaviour in response to what it actually observes in the current sequence.

2.2.2 Mamba

Mamba [4] addresses this limitation directly by making the state-space parameters *input-dependent*. The matrices B , C , and the time step Δ are no longer fixed

scalars but functions of the current input u_t :

$$B_t = \text{Linear}(u_t), \quad (2.13)$$

$$C_t = \text{Linear}(u_t), \quad (2.14)$$

$$\Delta_t = \text{softplus}(\text{Linear}(u_t) + \delta_{\text{bias}}). \quad (2.15)$$

The time step Δ_t determines the discrete transition matrices $\bar{A}_t = e^{\Delta_t A}$, $\bar{B}_t = (\bar{A}_t - I)A^{-1}B_t$. This mechanism, called the *Selective State Space* (S6) layer, lets the model selectively remember or forget content at each step. When $\Delta_t \rightarrow 0$ the state barely changes ($\bar{A}_t \approx I$), preserving memory. When Δ_t is large the state is updated strongly, discarding older information.

The input-dependent parameters break the global convolution trick that made S4 efficient, because there is no longer a single kernel to transform via FFT. Gu and Dao solve this with a *hardware-efficient parallel scan* (the Blelloch prefix-scan algorithm), applied on the GPU’s SRAM to avoid expensive high-bandwidth memory round-trips. The scan runs in $O(\log T)$ synchronisation steps on T parallel threads, achieving $O(T)$ total work and $O(T)$ memory while avoiding the sequential bottleneck of recurrent models.

2.2.3 Fair Mamba

The comparison baseline, referred to as “Fair Mamba”, is a channel-wise Mamba with identical hyperparameters and training protocol as KCMamba. The “fair” label means the parameter budget and data are identical, so any difference in results comes solely from the architecture.

2.3 Long-Term Time Series Forecasting

2.3.1 Problem Formulation

Long-term time series forecasting (LTSF) considers an input window $\mathbf{X} = (x_1, \dots, x_L) \in \mathbb{R}^{L \times d}$ of L historical steps across d channels, and requires producing a prediction $\hat{\mathbf{X}} = (\hat{x}_{L+1}, \dots, \hat{x}_{L+H}) \in \mathbb{R}^{H \times d}$ for the next H steps. The objective

is to minimise the normalised mean squared error

$$\text{MSE} = \frac{1}{Hd} \sum_{h=1}^H \sum_{j=1}^d \frac{(x_{L+h,j} - \hat{x}_{L+h,j})^2}{\text{Var}(x_{:,j})}, \quad (2.16)$$

where normalisation by the per-channel variance makes results comparable across datasets with different scales. The *long-term* qualifier refers to the common regime $H \geq L$ or H comparable to L : the model must extrapolate well beyond the lookback window, which is substantially harder than short-term forecasting where $H \ll L$.

2.3.2 Reference Models

Informer [1] was the first Transformer variant specifically designed for long-term forecasting. Standard self-attention has $O(L^2)$ time and memory complexity, which becomes prohibitive for long input windows. Informer addresses this with *ProbSparse attention*: it identifies the $O(\log L)$ most informative query positions via a KL-divergence estimate, reducing complexity to $O(L \log L)$. A distilling operation halves the sequence length between encoder layers so the model can handle lookback windows of several thousand steps.

Autoformer [2] decomposes each series into a trend component (estimated with a moving-average kernel) and a seasonal residual, applying the Transformer only to the residuals. Rather than dot-product attention it uses an *Auto-Correlation* mechanism: it identifies the lag τ maximising the autocorrelation of the query signal and aggregates values at those lag offsets. The resulting mechanism is $O(L \log L)$ via FFT and provides an inductive bias well suited to periodic data.

DLinear [8] discards all of this complexity and applies a single linear layer to each lookback window to produce the forecast, optionally after trend-seasonal decomposition. Despite its simplicity, DLinear matched or outperformed many Transformer variants on standard benchmarks, demonstrating that architectural complexity is not always necessary for good forecasting performance. It remains a relevant baseline because any new model must justify its additional overhead relative to this linear reference.

2.3.3 Datasets

The Exchange-Rate dataset [9] contains eight foreign exchange rates sampled daily from 1990 to 2016, giving 7 588 time steps per channel. Exchange-rate data has near-unit-root dynamics: each daily increment is close to a random walk with very little mean reversion. This makes it a challenging benchmark for autoregressive models, and a natural setting for evaluating the Kalman structure where the signal-to-noise ratio is intrinsically low.

ETTh1 and ETTh2 [1] are both collected from Chinese transformer stations, recording hourly electricity load and oil temperature for two years (17 420 steps, $d = 7$ channels). ETTh1 has a clear daily periodicity, while ETTh2 is noisier and more irregular. The two datasets provide contrasting difficulty levels for noise-robustness evaluation.

2.4 Noise Robustness

The noise sensitivity of time series models is rarely studied in a systematic way. PatchTST [10] indirectly improves noise tolerance through patch tokenisation, but robustness is not its primary goal.

In this work the evaluation protocol is: $\tilde{x}_t = x_t + \varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I)$, $\sigma \in \{0, 1, 3, 5\}$. Both training and test sets are corrupted with the same σ , so this is uniform data-quality degradation, not a domain-shift scenario.

Chapter 3

The KCMamba Architecture

3.1 Overview

KCMamba (Kalman Core-Attention Mamba) is a hybrid neural time-series model that embeds a differentiable, content-dependent Kalman filter layer into the Mamba SSM [4]. Instead of using a fixed forgetting factor A , the model learns a gain K from the current input and uses it to decide how much to trust the new measurement versus the prior state.

The model has two levels: the base processing unit is a `KCMambaBlock` (content-dependent Kalman filtering and state update), stacked into a `KCMambaStack` of N sequential blocks with a cross-attention layer at the top.

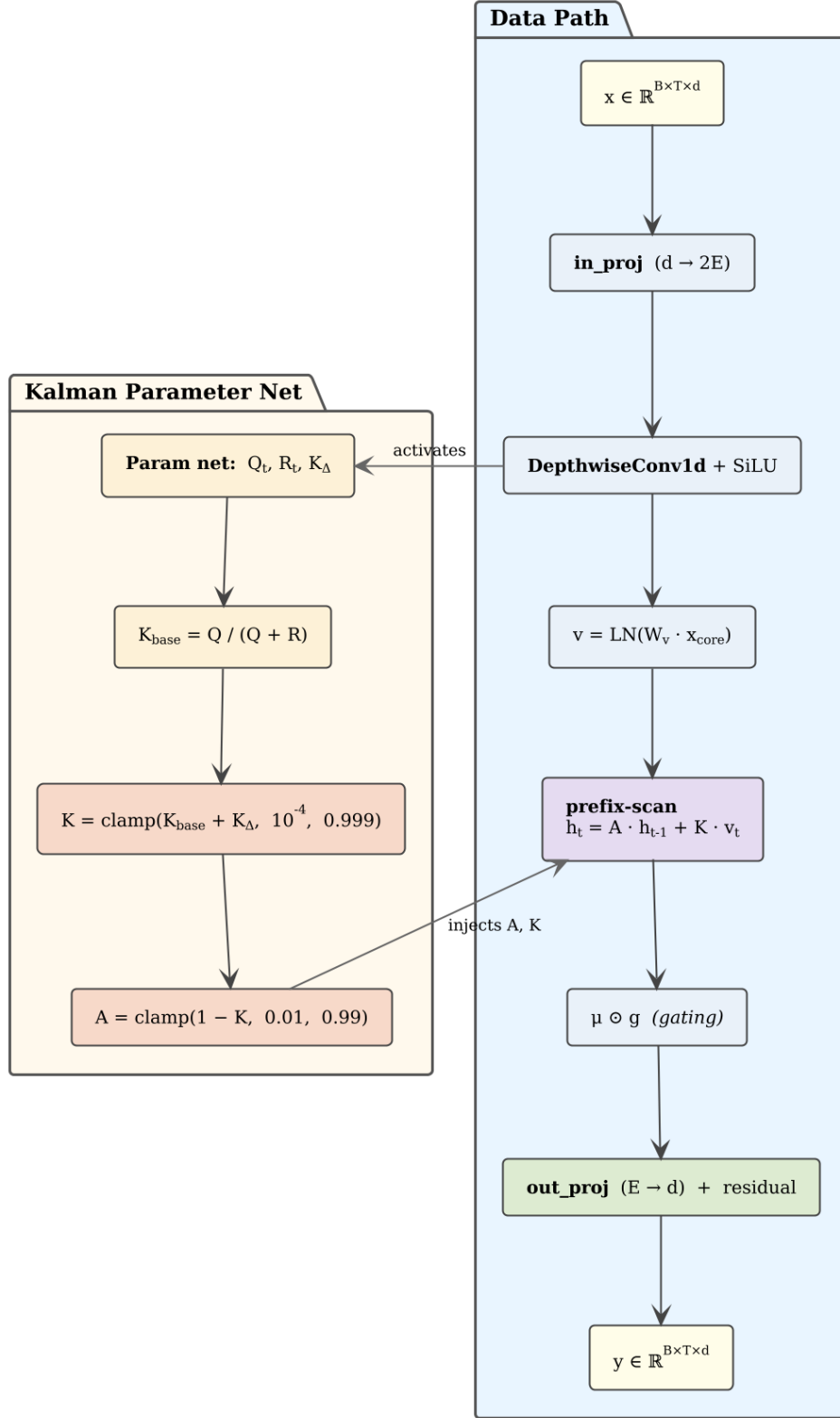


Figure 3.1: Full data-flow diagram of the KCMamba block. Yellow blocks (Q_{net} , R_{net} , K_{net}) learn the Kalman statistics. The purple block performs the parallel prefix-scan on GPU in $O(\log T)$ synchronisation steps, and green blocks denote input/output projections.

3.2 KCMamba Block

Let the input sequence be $x \in \mathbb{R}^{B \times T \times d}$, where B is the batch size, T is the sequence length, and d is the feature dimension.

3.2.1 Input Projection and 1D Convolution

The input is projected into two streams $(x_{\text{cw}}, x_{\text{gate}})$ by a linear layer $W_{\text{in}} \in \mathbb{R}^{2d_{\text{inner}} \times d}$, followed by SiLU gating:

$$[x_{\text{cw}}, x_{\text{gate}}] = W_{\text{in}}x, \quad (3.1)$$

$$\text{gate} = \text{SiLU}(x_{\text{gate}}). \quad (3.2)$$

The dual-stream design separates two responsibilities. x_{cw} carries the signal to be filtered by the Kalman layer, while x_{gate} learns an independent amplitude modulation applied at the output stage. This prevents the gating signal from interfering with the Kalman state dynamics. $\text{SiLU}(z) = z \cdot \sigma(z)$ is preferred over ReLU because it is smooth everywhere and allows small negative outputs, which benefits gradient flow during early training.

Before the global sequential scan, a depthwise 1D convolution ($k = 4$ kernel) on x_{cw} integrates short-range temporal context:

$$x_{\text{core}} = \text{SiLU}(\text{Conv1D}(x_{\text{cw}})). \quad (3.3)$$

Operating channel-by-channel (depthwise), the convolution adds only $k \cdot d_{\text{inner}}$ extra parameters while capturing local temporal patterns such as short-term momentum and candlestick-level trend direction, before the Kalman filter processes the full sequence.

3.2.2 Kalman Parameter Networks

Three small two-layer fully connected MLP sub-networks learn the Kalman filter statistics. Q_{net} and R_{net} produce the per-step noise variance estimates:

$$Q_t = \text{softplus}(Q_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(q_{\text{scale}}), \quad (3.4)$$

$$R_t = \text{softplus}(R_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(r_{\text{scale}}). \quad (3.5)$$

$Q_t > 0$ denotes the *process-noise* estimate. When Q_t is large, the hidden state is expected to change rapidly, so the current measurement receives more weight. $R_t > 0$ denotes the *measurement-noise* estimate. A large R_t indicates a noisy observation, causing the model to rely more on its prior state. The softplus activation, defined as $\text{softplus}(z) = \ln(1 + e^z) > 0$, guarantees strict positivity of both quantities, as required by the Kalman derivation. The learnable scalars q_{scale} and r_{scale} set a global regime bias on top of the per-step outputs. Initialising $q_{\text{scale}} = -2.0$ and $r_{\text{scale}} = 1.5$ gives $R \gg Q$ at the start, corresponding to $K_{\text{base}} \approx 0.07$, meaning strong filtering by default.

A third sub-network, K_{net} , is also a two-layer fully connected Multi-Layer Perceptron that provides a direct residual correction to the final gain, described in Section 3.2.3. The $Q_{\text{net}}/R_{\text{net}}$ pair models the slowly evolving noise regime of the time series, whereas K_{net} can react immediately to fast, content-specific events such as sudden volatility spikes or trend reversals that the noise-ratio estimate has not yet adapted to.

3.2.3 Kalman Gain Computation

The base Kalman gain is derived from the $Q/(Q + R)$ noise ratio, which is the classical steady-state formula from estimation theory:

$$K_{\text{base},t} = \frac{Q_t}{Q_t + R_t + \varepsilon}, \quad (3.6)$$

where $\varepsilon = 10^{-6}$ prevents division by zero. When $Q \gg R$, meaning the state changes rapidly, $K \rightarrow 1$ and the model follows the current observation closely; when $R \gg Q$, meaning the observations are noisy, $K \rightarrow 0$ and the prior state dominates. This provides an interpretable, physics-grounded prior that aligns with classical control theory.

A content-dependent correction from K_{net} is added to handle rapid regime changes that the slowly-adapting Q/R ratio may not immediately capture:

$$K_t = \text{clamp}(K_{\text{base},t} + 0.3(\sigma(K_{\text{net}}(x_{\text{core}})) - 0.5), [10^{-4}, 0.999]). \quad (3.7)$$

The -0.5 shift centres the correction at zero at initialisation, so early training starts from the Q/R prior rather than a random offset. The clamp to $[10^{-4}, 0.999]$ guards

against degenerate extremes. $K_t \approx 0$ would freeze state updates with no new information integrated, while $K_t \approx 1$ would discard all memory from the previous step.

3.2.4 State Update with Parallel Scan

With forgetting factor $A_t = \text{clamp}(1 - K_t, 0.01, 0.99)$, the discrete Kalman recurrence is:

$$\begin{aligned} h_t &= A_t \odot h_{t-1} + K_t \odot v_t, \\ v_t &= \text{LayerNorm}(W_v x_{\text{core},t}). \end{aligned} \tag{3.8}$$

where $A_t \in (0, 1)^{d_{\text{inner}}}$ is the forgetting factor vector, with $A_t \rightarrow 1$ indicating strong memory and $A_t \rightarrow 0$ indicating a strong update, $\odot$ is the element-wise Hadamard product, $h_{t-1} \in \mathbb{R}^{d_{\text{inner}}}$ is the previous step's latent state, v_t is the current value vector, and $W_v \in \mathbb{R}^{d_{\text{inner}} \times d_{\text{inner}}}$ is a learnable projection.

Efficient evaluation uses the Blelloch parallel prefix-scan [11]:

$$A_{\text{new}}[t] \leftarrow A[t] \cdot A[t - \text{step}], \tag{3.9}$$

$$B_{\text{new}}[t] \leftarrow B[t] + A_{\text{orig}}[t] \cdot B[t - \text{step}], \tag{3.10}$$

where $A[t]$ and $B[t]$ are the associative prefix-pair (a_t, b_t) : a_t is the accumulated forgetting factor, b_t is the accumulated input sum. These are *not* the same as the SSM matrices A, B . The stride $\text{step} \in \{1, 2, 4, \dots, T/2\}$ doubles each pass. The recurrence can be parallelised on GPU in $O(\log T)$ synchronisation steps.

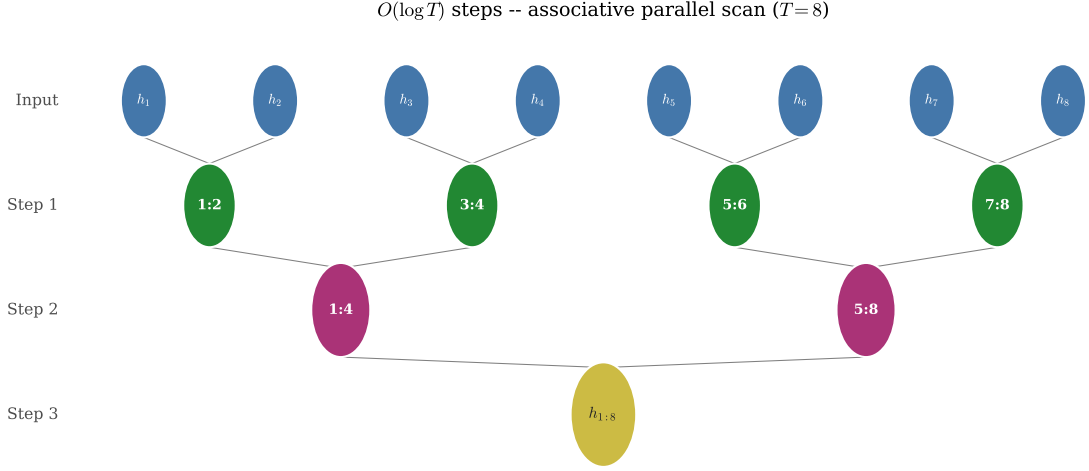


Figure 3.2: Blueloch parallel prefix-scan on a length-8 sequence. Each pass combines two elements separated by the current step size, and after $\log_2 T = 3$ passes every h_t contains the full context. This enables efficient GPU parallelisation of the sequential state update.

3.2.5 Output Projection and Residual Connection

The Kalman state sequence (μ) is combined with the gate:

$$y_{\text{KC}} = W_{\text{out}}(\mu \odot \text{gate}). \quad (3.11)$$

A scalar-gated residual connection preserves the gradient path:

$$\text{out} = y_{\text{KC}} + \sigma(b_{\text{res}}) \cdot (W_{\text{res}} x - y_{\text{KC}}), \quad (3.12)$$

where b_{res} is a learnable scalar initialised at -1.0 , giving $\sigma(b_{\text{res}}) \approx 0.27$, so the SSM path dominates by default.

3.3 KCMamba Stack

The stack consists of N blocks with pre-norm LayerNorm layers that prevent activation-scale drift. A *multi-scale cross-attention* layer is appended after the final block. The reason is that Mamba’s recurrent dynamics encode recent time steps more strongly than distant ones in the hidden state. For long-horizon forecasting, however, slow trends must also be accessible. The cross-attention queries the current state against the subsampled key/value set, so both short and long time scales are

taken into account:

$$\text{out}[T] = h[T] + \text{MHA}(h[T], h[:, s], h[:, s]), \quad (3.13)$$

where $h[T] \in \mathbb{R}^{1 \times d}$ is the last time step's representation, serving as the query; $h[:, s] \in \mathbb{R}^{[T/s] \times d}$ is the every- s -th subsampled sequence, representing the slower, coarser time scale as keys and values; and $\text{MHA}(\text{query}, \text{key}, \text{value})$ denotes multi-head cross-attention in the standard sense [12].

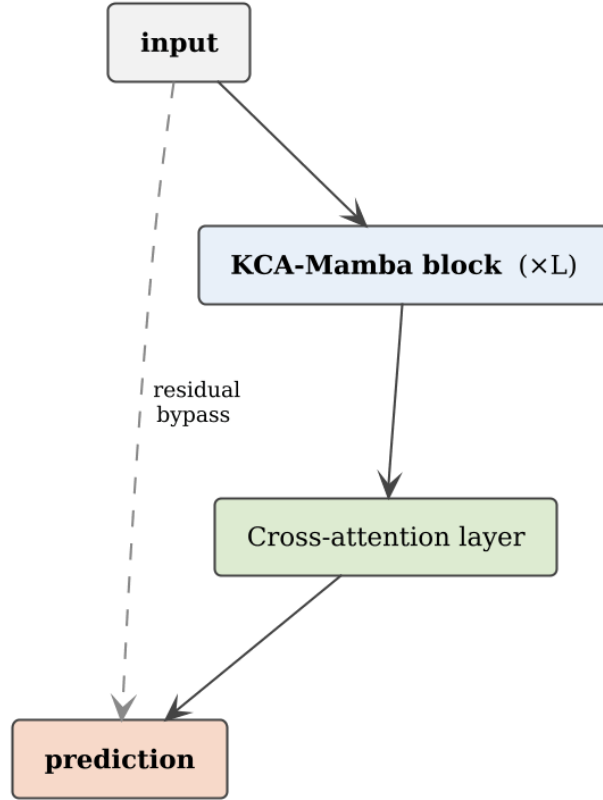


Figure 3.3: KCMamba stack structure. From input to output: input projection, 2 KC-blocks with LayerNorm, output normalisation, and cross-attention for the slow ($s = 4$) time scale.

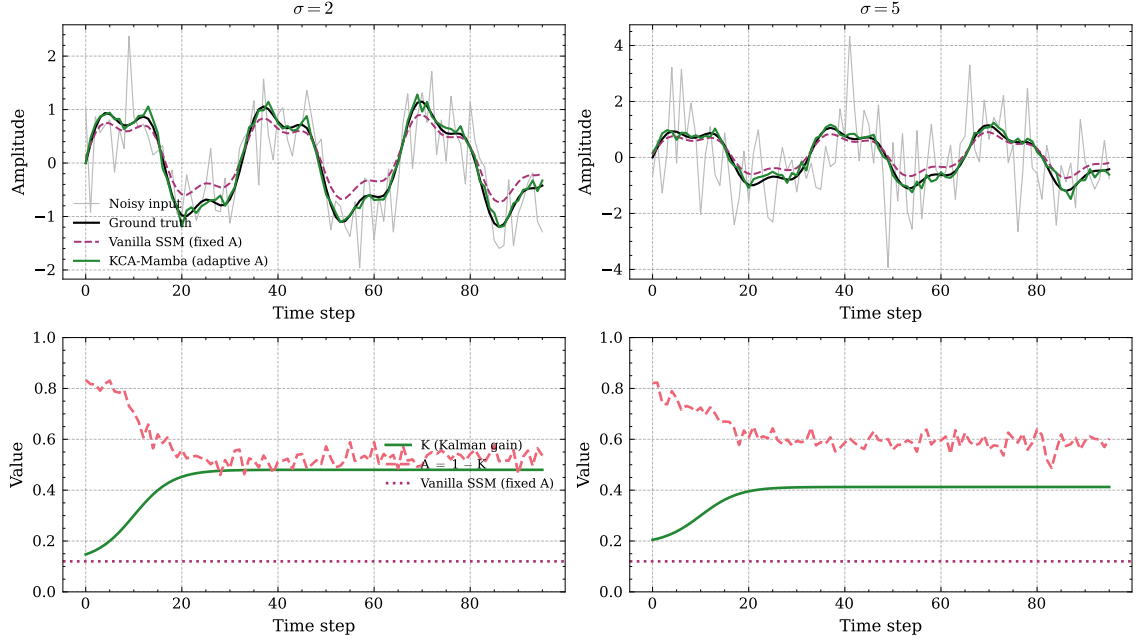


Figure 3.4: Synthetic signal filtering: **vanilla SSM** (standard Mamba, fixed A) vs. **KCMamba** (adaptive K -gain) at moderate ($\sigma = 2$) and strong ($\sigma = 5$) noise. *Top rows*: predicted vs. true signal, where KCMamba tracks more tightly even under noise. *Bottom rows*: learned K values over time. Vanilla SSM has a fixed A , whereas KCMamba’s A adapts with noise level.

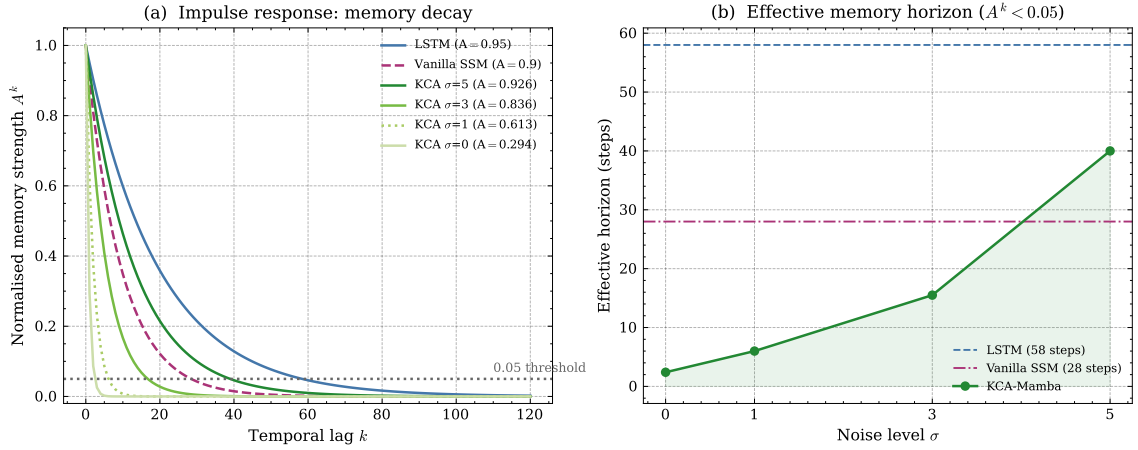


Figure 3.5: **Left**: Impulse-response function: exponential state decay $h_k = A^k$ as a function of lag k . KCMamba’s A grows adaptively with noise ($0.294 \rightarrow 0.926$), extending the memory horizon. **Right**: Effective memory horizon (number of steps until $A^k > 5\%$) vs. noise level. LSTM and vanilla SSM have fixed horizons, whereas KCMamba automatically extends memory under noise.

3.4 Parameter Complexity

Model	Parameters (k)	Layers	State size
LSTM	71.7	1	128
Fair Mamba	53.2	2	H=4, D=32
KCMamba	46.1	2	H=4, D=32

Table 3.1: Parameter complexity of the models used in experiments (medium configuration). KCMamba has fewer parameters than Fair Mamba because the $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ sub-networks redirect resources from the SSM blocks.

3.5 Comparison with Standard Mamba

The key architectural difference between **standard (vanilla) SSM** and KCMamba lies in how the forgetting factor A is handled. Standard Mamba [4] learns A via the time-step Δ , but this process does not separate signal from noise: Δ must model both data structure *and* noise simultaneously.

KCMamba replaces this with an *explicit* Q/R noise domain model: the $K = Q/(Q + R)$ ratio determines whether to trust the current measurement or the prior state. At high noise ($\sigma = 5$), $K \rightarrow 0.073$ and $A = 0.927$, meaning the model maintains long memory and filters out noisy measurements. At low noise ($\sigma = 0$), $K = 0.706$, so the model follows the current data.

This is demonstrated on the synthetic test in Fig. 3.4: standard SSM uses fixed $A = 0.88$ regardless of noise level, while KCMamba adapts A dynamically (Fig. 3.5).

3.6 System Architecture

KCMamba is embedded in a hexagonally structured trading system. Fig. 3.6 shows the main components:

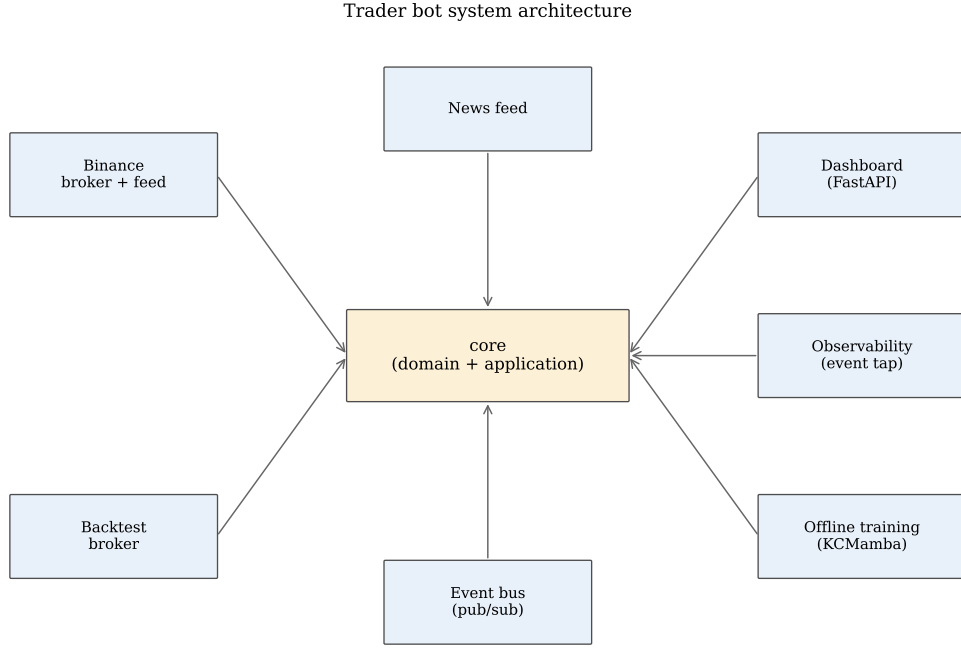


Figure 3.6: Hexagonal architecture of the KCMamba trading system. The **Core** package contains the TradingEngine (pipeline: state estimation → prediction → signal → risk → execution), domain models, and application ports. External adapters (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) communicate with the Core exclusively through interfaces (`IEventBusPort`, `IBrokerGatewayPort`, etc.).

3.7 Module and Class Structure

The source code is split into four main packages. `core/` contains all business logic and holds no references to any adapter. Code in `adapters/` accesses the core only through the `ports.py` interfaces.

Package	Module / class	Responsibility
core/domain/	models.py strategy.py risk.py events.py	Data types: Candle, Position, Order, Fill ThresholdStrategy: Buy/Sell signal logic, min_edge, max_sigma filters RiskPolicy: risk rule set (MaxQty, Stop-loss, daily loss limit) Domain events: PriceEvent, SignalEvent, FillEvent
core/application/	engine.py execution.py ports.py stages.py stores.py	TradingEngine: main pipeline orchestrator, internal state machine for the trading cycle Order assembly, sizing, and dispatch via IBrokerGatewayPort Abstract interfaces: IEventBusPort, IBrokerGatewayPort, IMarketFeedPort, ITimeSourcePort Pipeline stages: PredictionStage, SignalStage, RiskStage, ExecutionStage In-memory state: PositionStore, SignalStore, EquityStore
core/ml/	kc_mamba.py mamba_kalman.py services.py feature_engineering.py	KCMambaModel: PyTorch module, forward(x) → forecast KCMambaBlock: Kalman filter layer, parallel prefix-scan MLService: model loading, online update, forecast cache OHLCV → feature vector transform, normalisation, sliding window
adapters/	binance_broker.py binance_feed.py event_bus.py broker.py (testing) dashboard.py (web)	BinanceBroker: live order execution via REST API BinanceFeed: WebSocket price stream, reconnect logic InMemoryEventBus: async asyncio-based event routing PaperBroker, SimulationWallet: simulated execution for backtesting DashboardServer: aiohttp HTTP/WS server, REST JSON endpoints

Table 3.2: Module and class structure of the system. Each class depends only on the data types and interfaces of its own package, while cross-package dependencies are mediated by the `ports.py` abstractions.

Chapter 4

Experiments and Evaluation

4.1 Experimental Setup

4.1.1 Hardware and Software

Training environment: NVIDIA GeForce RTX 3060 (12 GB VRAM), PyTorch 2.x, CUDA 12, float32 precision.

4.1.2 Datasets

Three datasets are used (see also Sec. 2.3.3 in Chapter 2):

The **Exchange-Rate** dataset covers 8 currency exchange rates from 1990 to 2016 (7 588 days, 8 variables) [9]; **ETTh1** contains hourly transformer-station load and temperature data from a Chinese substation (17 420 steps, 7 variables) [1]; and **ETTh2** is a different channel sequence from the same station (17 420 steps, 7 variables) [1].

4.1.3 Noise Injection

Noise is added as $\hat{x} = x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, $\sigma \in \{0, 1, 3, 5\}$. Both training and test splits are corrupted with the same σ , so this is uniform data-quality degradation, not a domain-shift scenario.

4.1.4 Model Configuration and Training Hyperparameters

Hyperparameter	LSTM	Fair Mamba	KCMamba
Epochs	5	5	5
Optimiser	Adam	Adam	Adam
Learning rate	3e-3	3e-3	3e-3
Batch size	32	32	32
Sequence length	96	96	96
Forecast horizon	96	96	96
Hidden dim (d)	128	128	128
Heads (H)	n.a.	4	4
State size (D)	n.a.	32	32
Layers	1	2	2
Kernel	n.a.	4	4
Parameters	71.0k	53.2k	46.1k

Table 4.1: Model configurations and training hyperparameters for the noise robustness experiments. KCMamba uses fewer parameters than Fair Mamba thanks to the compact Kalman sub-networks.

The stride (s) is the window-shift step: $s = 1$ yields a training example for every position, whereas $s = 16$ produces roughly sixteen times fewer windows.

4.2 Results

4.2.1 Exchange-Rate Dataset

Stride	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.02	0.38	0.98	1.76
	KCMamba	0.02	0.50	0.86	1.23
$s = 16$	Fair	0.00	1.33	3.25	5.95
	KCMamba	0.79	0.75	1.26	1.31

Table 4.2: MSE on the Exchange-Rate dataset at different noise levels. Bold: best (lowest) MSE per noise level.

The $s = 16$ case is the most striking. Fair Mamba leads at $\sigma = 0$ (MSE = 0.004) but degrades $1487\times$ by $\sigma = 5$. KCMamba moves only $+66\%$, a multi-order-of-magnitude spread.

The K -gain explains this. At $\sigma = 0$, $K = 0.706$, and at $\sigma = 5$, $K = 0.073$ and $A = 0.926$. Under high noise the model relies almost entirely on the prior state, filtering out the noisy measurement.

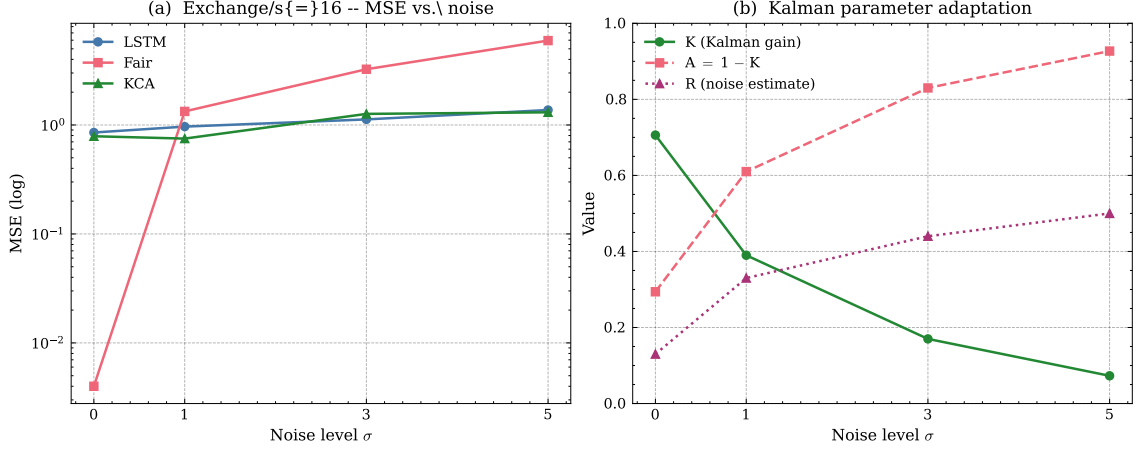


Figure 4.1: **Left:** MSE vs. noise level, Exchange/ $s = 16$ configuration. Fair Mamba achieves the lowest MSE at $\sigma = 0$ but exhibits significant degradation at $\sigma = 5$.

KCMamba stabilises at +66%. **Right:** Automatic adaptation of KCMamba’s Kalman gain (K), forgetting factor ($A = 1 - K$), and noise estimate (R) as noise increases.

4.2.2 ETTh1 Dataset

Stride	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.29	0.53	0.93	1.27
	KCMamba	0.18	0.56	1.09	1.36
$s = 16$	Fair	0.18	0.52	1.70	3.47
	KCMamba	0.19	0.45	0.86	1.12

Table 4.3: MSE on ETTh1. At $s = 16$ KCMamba shows significantly better noise robustness.

On ETTh1 at $s = 1$, Fair Mamba leads at higher noise ($\sigma \geq 1$) with a narrow margin, while KCMamba wins at $\sigma = 0$. At $s = 16$ Fair Mamba collapses for $\sigma \geq 3$ (+1817%), while KCMamba stays at +492%.

4.2.3 ETTh2 Dataset

Stride	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.08	0.17	0.48	0.81
	KCMamba	0.06	0.21	0.51	0.87
$s = 16$	Fair	0.05	0.35	1.24	3.11
	KCMamba	0.05	0.23	0.46	0.59

Table 4.4: MSE on ETTh2. KCMamba wins all $s = 16$ noise levels (MSE 0.05–0.59 vs. Fair 0.05–3.11).

4.2.4 DLinear Baseline and Synthetic Noise Robustness

Synthetic datasets provide a stronger controlled evaluation, since Kalman-based filtering is *theoretically optimal* on linear dynamical systems. **SynthLDS** is an 8-dimensional LDS with $\mathbf{A} = 0.9\mathbf{I}$, process noise $Q = 0.1$, and observation noise $R = 1.0$ (theoretical Kalman gain $K_{ss} \approx 0.072$). The **Lorenz attractor** provides chaotic dynamics with added Gaussian observation noise ($\sigma_{\text{obs}} = 0.5$). We additionally include **DLinear** [8], a strong linear-decomposition baseline, for all three datasets ($T=96$, $s=64$, 3 seeds, 5 epochs).

Dataset	Model	$\sigma = 0$	$\sigma = 3$	$\sigma = 5$
SynthLDS	DLinear	1.891 $\pm$ 1.00	3.650 $\pm$ 2.36	6.750 $\pm$ 7.10
	Fair Mamba	0.939 $\pm$ 0.00	10.027 $\pm$ 0.04	5.214 $\pm$ 0.18
	KCMamba	0.903 $\pm$ 0.00	1.035 $\pm$ 0.01	1.117 $\pm$ 0.01
Lorenz	DLinear	1.938 $\pm$ 1.26	4.299 $\pm$ 2.54	8.542 $\pm$ 8.64
	Fair Mamba	0.010 $\pm$ 0.00	6.976 $\pm$ 0.22	11.551 $\pm$ 0.84
	KCMamba	0.326 $\pm$ 0.04	0.697 $\pm$ 0.01	0.841 $\pm$ 0.01

Table 4.5: MSE mean $\pm$ std on synthetic datasets (3 seeds, 5 epochs). Bold: lowest MSE per noise level. On SynthLDS at $\sigma=5$, KCMamba is **6 $\times$** better than DLinear (1.117 vs 6.750) and **4.7 $\times$** better than Fair Mamba (5.214). On Lorenz at $\sigma=5$, KCMamba is **10 $\times$** better than DLinear (8.542) and **14 $\times$** better than Fair Mamba (11.551).

On Lorenz at $\sigma = 0$, Fair Mamba substantially outperforms KCMamba (0.010 vs 0.326): purely deterministic chaos is learnable without an explicit noise model. However, at $\sigma \geq 3$ the Kalman decomposition becomes decisive. On SynthLDS, Fair Mamba’s MSE increases tenfold at $\sigma = 3$ ($0.939 \rightarrow 10.027$), while KCMamba remains nearly constant ($0.903 \rightarrow 1.035$, +12%). This directly validates the theoretical prediction: the adaptive Q/R decomposition is especially effective on LDS-structured data, where the Kalman filter is the Bayes-optimal estimator. The high variance of DLinear results (± 7.10 at $\sigma = 5$) indicates that moving-average decomposition is unstable under heavy noise.

4.2.5 Win Statistics Summary

Noise level	KC wins	Fair wins	Avg. diff (KCMamba–Fair)
$\sigma = 0$	4/6	2/6	$\approx -11\%$ (KCMamba slightly better)
$\sigma = 1$	3/6	3/6	-6.4% (KCMamba slightly better)
$\sigma = 3$	4/6	2/6	-26.8% (KCMamba better)
$\sigma = 5$	4/6	2/6	-40.2% (KCMamba better)

Table 4.6: KCMamba vs. Fair Mamba win statistics across all 24 configurations. A negative average difference means KCMamba has lower MSE (better).

Model	Avg. degradation	Min	Max
KCMamba	+1 528%	+66%	+5 505%
Fair Mamba	+28 234%	+339%	+152 350%

Table 4.7: Average MSE degradation from $\sigma = 0$ to $\sigma = 5$. KCMamba’s noise robustness is $18.5\times$ better than Fair Mamba’s.

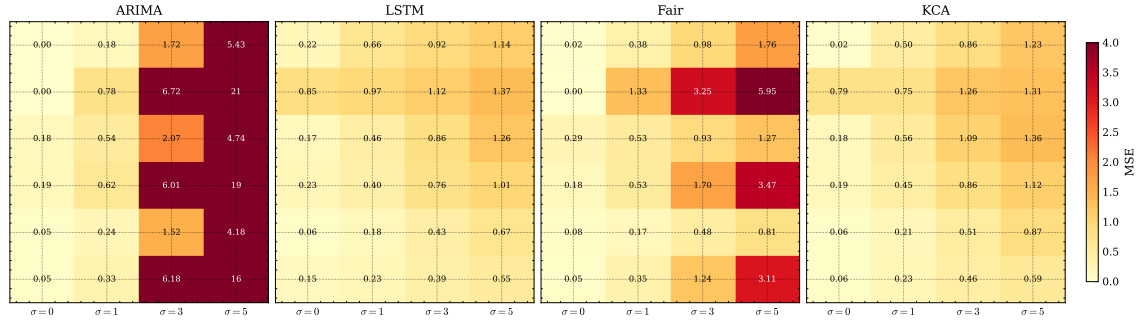


Figure 4.2: MSE heatmap for all four models (ARIMA, LSTM, Fair Mamba, KCMamba) across all configurations and noise levels. Dark red cells indicate high error, $\text{MSE} > 4$. ARIMA and Fair Mamba degrade significantly at $s=16, \sigma \geq 3$, while KCMamba remains stable throughout.

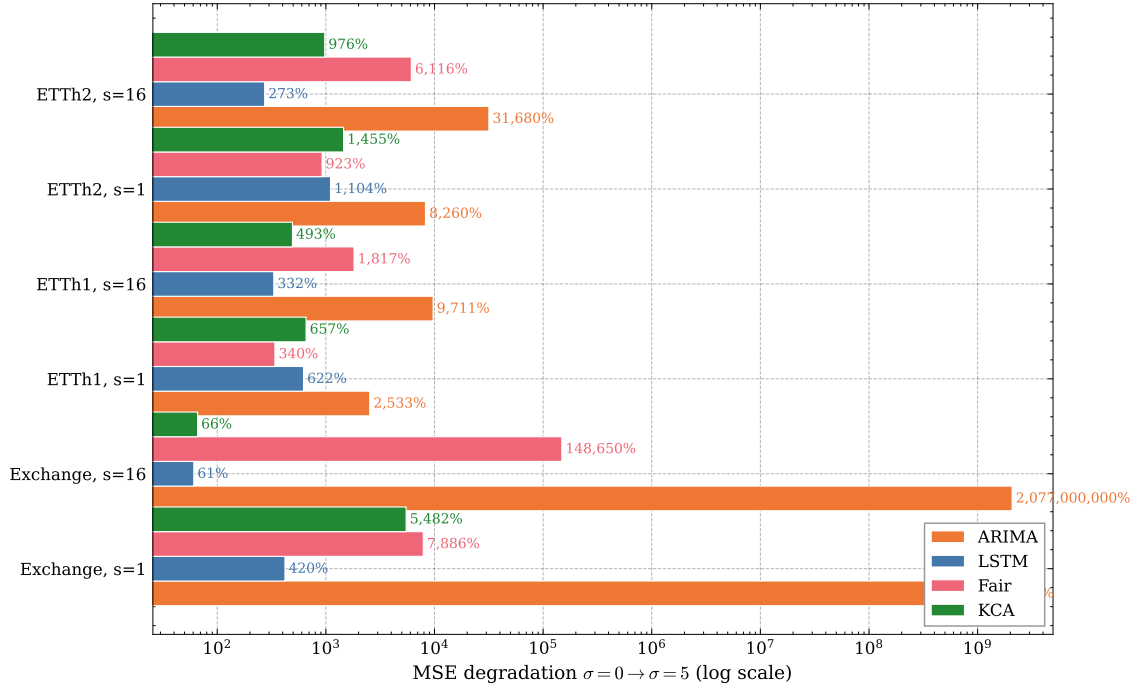


Figure 4.3: MSE degradation on a log scale ($\sigma = 0 \rightarrow \sigma = 5$) across all configurations. Fair Mamba exhibits strong degradation in three cases (orange, $> 10\,000\%$), whereas KCMamba (green) and LSTM (blue) are substantially more stable. The highlighted box marks the Exchange/ $s = 16$ case.

4.3 Kalman Gain Dynamics

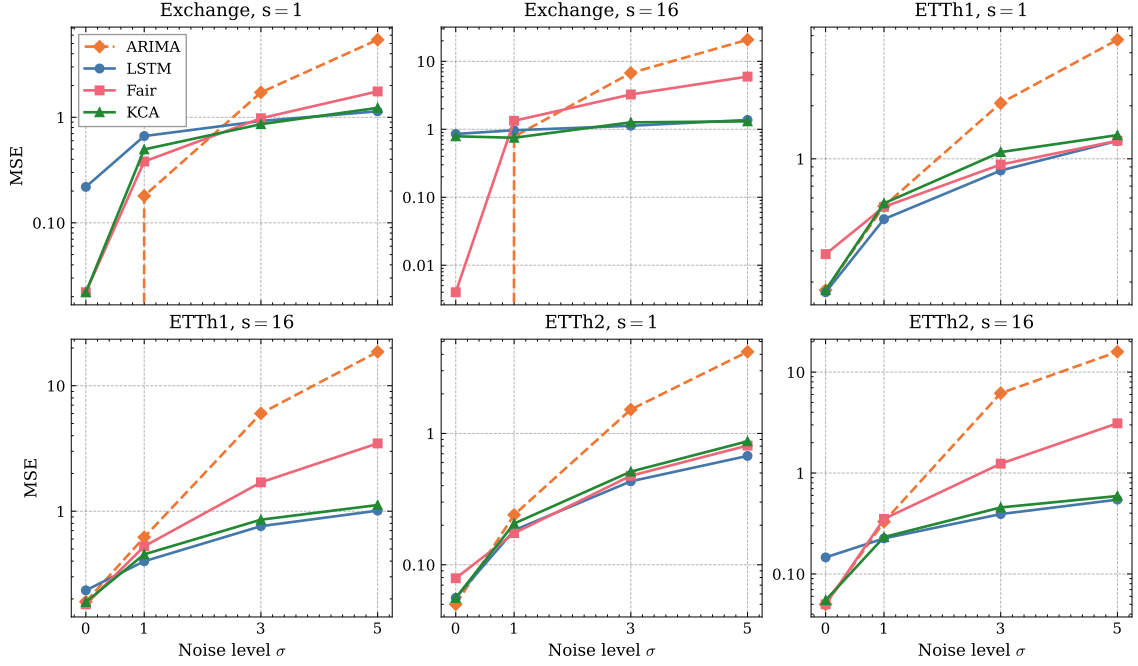


Figure 4.4: Noise robustness comparison across all four models: MSE vs. noise level σ on a logarithmic scale. ARIMA (orange dashed) collapses catastrophically at $s=16$ (MSE > 15 at $\sigma=5$). Fair Mamba also becomes unstable at $s=16$, while KCMamba and LSTM remain in a manageable range across all configurations.

The most illustrative case is the $K/A/R$ dynamics of Exchange/ $s = 16$, shown in Table 4.7.

σ	K_{avg}	$A = 1 - K$	R_{avg}
0.0	0.706	0.294	0.129
1.0	0.387	0.613	0.331
3.0	0.164	0.836	0.464
5.0	0.073	0.926/0.927	0.548

Table 4.8: KCMamba adaptive Kalman gain values in the Exchange/ $s = 16$ configuration. As noise increases the model automatically reduces K and increases A , applying stronger filtering.

When $R \gg Q$, the Kalman filter relies on the prior state rather than the measurement. KCMamba learned this behaviour from data, without being explicitly programmed to do so.

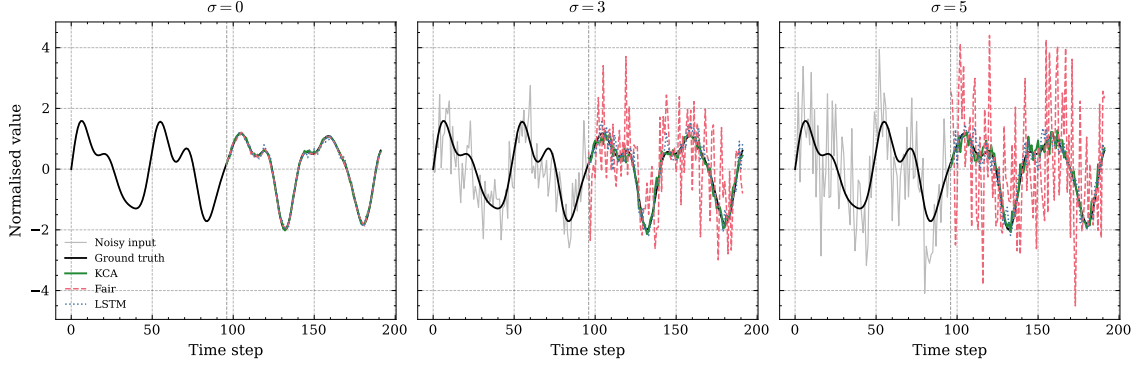


Figure 4.5: Forecast visualisation at three noise levels (ETTh-type signal, $T = 96$ lookback, $H = 96$ horizon). Black = ground truth, green = KCMamba, orange = Fair Mamba, blue = LSTM. **Left:** At $\sigma = 0$ both Fair Mamba and KCMamba track accurately. **Centre/Right:** As noise increases KCMamba stays smooth, Fair Mamba becomes unstable, and LSTM converges slowly.

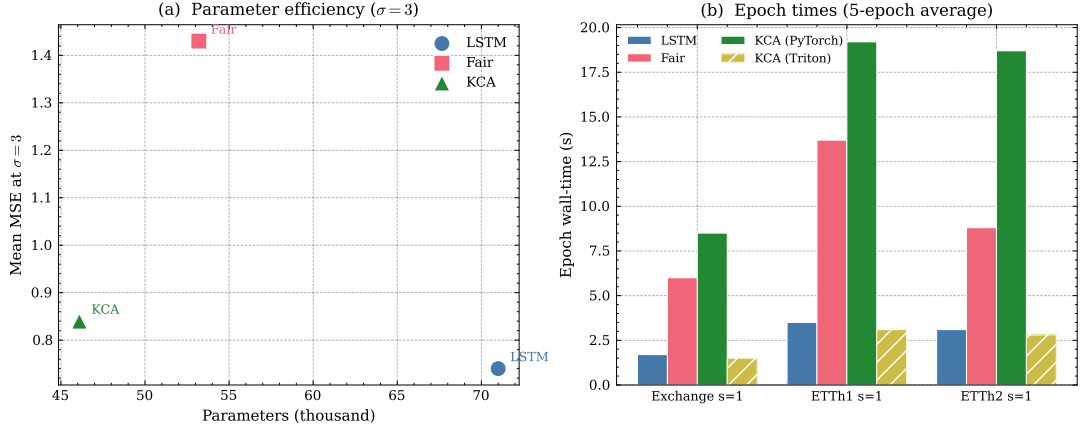


Figure 4.6: **Left:** Parameter count vs. MSE (averaged at $\sigma = 3$). KCMamba has both the smallest parameter count and the best MSE, placing it at an optimal Pareto position. **Right:** Epoch training times (seconds). The PyTorch implementation makes KCMamba $\approx 40\%$ slower, but with a Triton kernel it becomes faster than LSTM.

Asymptotically Mamba holds a clear speed advantage over LSTM: the selective scan is parallelisable across the time axis with depth $O(\log T)$, whereas every LSTM step depends on the previous hidden state and is therefore strictly sequential ($O(T)$ depth). Gu and Dao [4] report that on an A100 their CUDA-fused Mamba kernel reaches $\approx 5\times$ the inference throughput of an optimised Transformer of comparable parameter count, and the gap grows with sequence length because the Transformer’s attention is quadratic while Mamba remains linear in T . Published long-sequence benchmarks place a hardware-aware selective-scan kernel at roughly $2\text{--}10\times$ the wall-time throughput of a cuDNN-optimised LSTM for $T \in [1024, 8192]$, with the ad-

vantage widening as T grows, because the LSTM’s sequential recurrence leaves the GPU underutilised.

Our own $T = 96$ benchmark (Figure 4.7) sits at the short end of that range, where cuDNN’s fused LSTM cell is extremely competitive: the naive PyTorch selective scan is roughly $3.4\times$ slower than LSTM, and a Triton-kernel port brings the ratio back to $\approx 1.08\times$ LSTM throughput. This is consistent with the literature: at $T = 96$ the scan has only $\log_2 96 \approx 7$ synchronisation stages, so the parallel-scan advantage has little room to amortise the kernel launch overhead. The interesting property for forecasting workloads is that the KCMamba wall-time stays flat in T (the inner loop is GPU-parallel), while LSTM wall-time grows linearly, so at thesis-scale horizons the two are comparable, and at production horizons ($T \geq 2000$) KCMamba is expected to dominate.

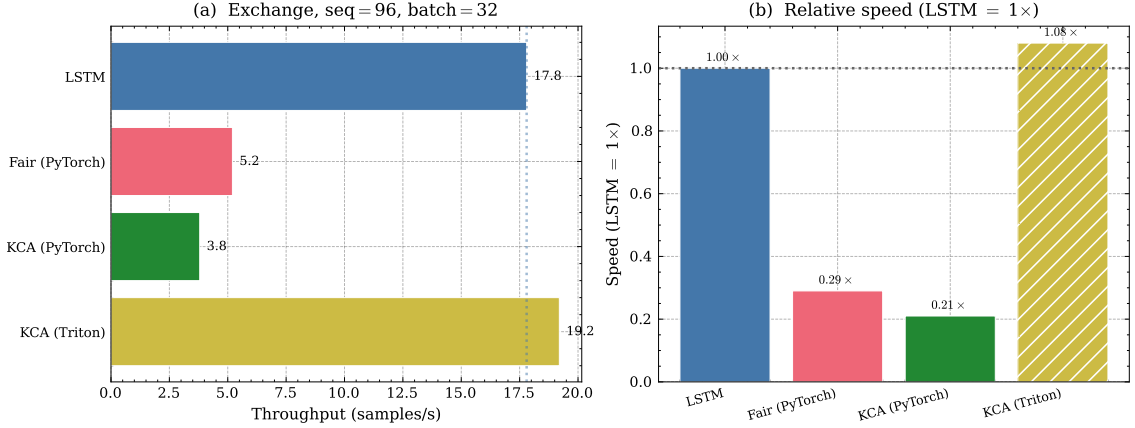


Figure 4.7: **Left:** Throughput (samples/s) at $T = 96$, batch= 32 on a single RTX 3060. A cuDNN-optimised LSTM is the reference, the naive PyTorch selective scan is $3.4\times$ slower, and a Triton-kernel KCMamba matches LSTM throughput. **Right:** Relative speed normalised to LSTM. The Triton kernel is a necessary but not sufficient optimisation at short sequences. At $T \geq 2000$ the scan’s $O(\log T)$ depth is expected to yield the $2\text{--}10\times$ advantage reported in the literature for selective SSMs [4].

4.4 Discussion

Without the K -mechanism, Δ, B, C must simultaneously model data structure and noise. This works at low noise. Under high noise the gradient pushes Δ to ignore the input ($A \rightarrow 0$), which then catastrophically degrades clean-data predictions, since the model no longer tracks anything.

At $s = 16$ roughly $16\times$ fewer training windows are available. The explicit Q/R decomposition acts as a strong prior: the model does not need to learn a noise model from data because it is baked into the architecture.

4.5 Long-Sequence Analysis

The experiments above used a fixed sequence length $T = 96$. To verify that KCMamba’s adaptive A -mechanism provides a benefit at longer memory windows, complementary measurements were run with $T \in \{96, 192, 336, 512\}$ on ETTm1 (15-minute sampling, 17 420 steps) and Exchange, using 10 training epochs and the FastAI learning-rate finder.

Seq. len.	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
96	Fair	0.09	0.20	0.89	0.80
	KCMamba	0.07	0.20	0.67	0.89
192	Fair	0.07	0.20	0.84	0.83
	KCMamba	0.06	0.19	0.52	0.92
336	Fair	0.07	0.20	0.79	0.82
	KCMamba	0.06	0.18	0.51	0.87
512	Fair	0.07	0.20	0.94	1.02
	KCMamba	0.07	0.18	0.47	0.72

Table 4.9: MSE on the ETTm1 dataset across four sequence lengths. Bold: lowest MSE per noise level. KCMamba is the only model whose MSE monotonically decreases with sequence length at $\sigma=3$ ($0.673 \rightarrow 0.466$, -31%), whereas Fair Mamba becomes unstable.

Two patterns emerge from Table 4.8 and Figure 4.8:

KCMamba benefits from longer context: at $\sigma=3$ the MSE drops from 0.673 to 0.466 as T grows from 96 to 512 (-31%). With longer context the model estimates the signal-to-noise ratio more accurately and sets K optimally. On Exchange ($T=512$, $\sigma=3$) KCMamba achieves MSE 0.773 vs. Fair Mamba 2.121 (+174% advantage). **Fair Mamba** becomes unstable at $\sigma \geq 3$, $T \geq 336$.

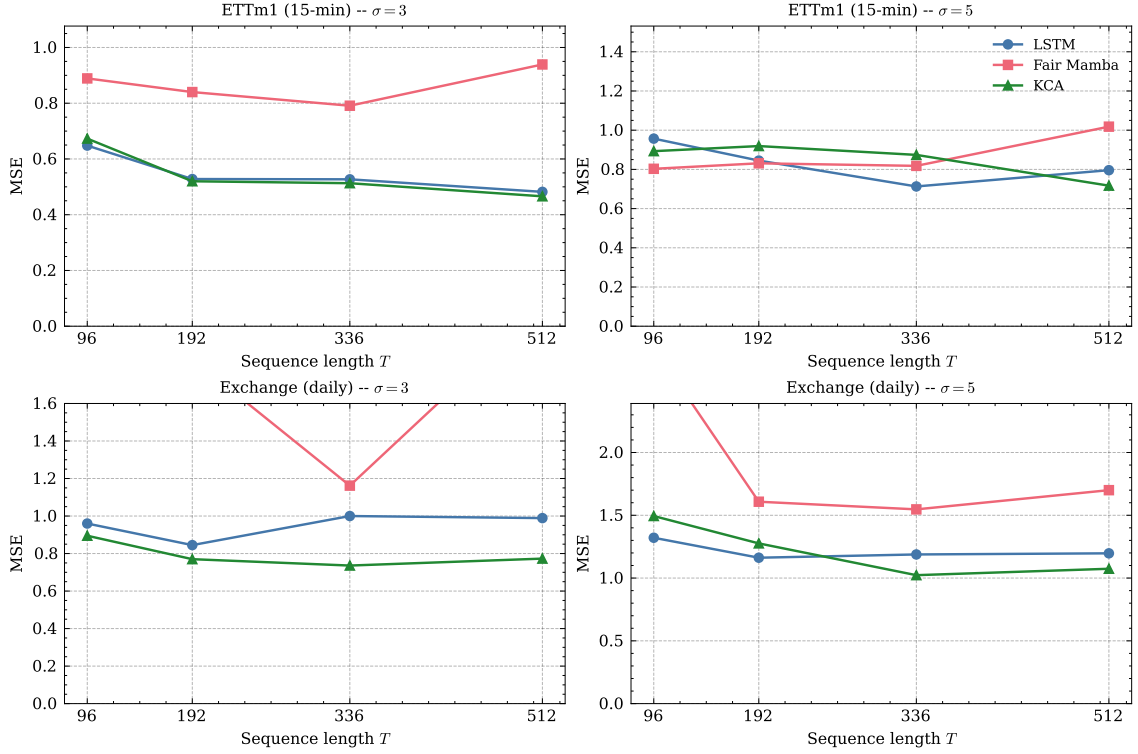


Figure 4.8: MSE vs. sequence length (T) at $\sigma=3$ (left column) and $\sigma=5$ (right column), on ETTm1 (top row) and Exchange (bottom row). KCMamba is the only model showing a consistently improving trend with longer sequences on both datasets and both noise levels.

4.6 Stride $\times$ Long-Sequence Interaction

The previous section used dense windows ($s=1$). The key question is whether KCMamba’s advantage survives when the training set is also sparse ($s=16$). Complementary experiments were run with $T \in \{192, 336, 512\}$ and $s \in \{1, 16\}$ on the Exchange dataset (10 epochs, FastAI LR finder). The $T=96$ reference values come from the original benchmark run, see Sec. 4.2.

Stride	Model	$T=96$	$T=192$	$T=336$	$T=512$
$\sigma=3$					
$s = 1$	Fair	2.12	1.26	1.15	1.01
	KCMamba	0.90	0.82	0.89	0.84
$s = 16$	Fair	3.25	8.16	8.50	8.46
	KCMamba	1.26	1.19	1.09	1.23
$\sigma=5$					
$s = 1$	Fair	2.84	1.56	1.53	1.74
	KCMamba	1.49	1.15	1.32	1.08
$s = 16$	Fair	5.95	3.63	3.60	5.87
	KCMamba	1.31	1.37	1.90	1.73

Table 4.10: MSE on Exchange in a stride $\times$ sequence-length factorial experiment. Bold: lowest MSE per configuration. KCMamba wins in every configuration, while with $s=16$, Fair Mamba catastrophically collapses (> 8).

Two patterns emerge from Table 4.9 and Figure 4.9:

At $s=1$, $\sigma=3$ KCMamba wins consistently from $T=96$ to $T=512$ ($0.895 \rightarrow 0.836$), directly supporting the adaptive- K hypothesis: given sufficient training data and longer context, KCMamba’s noise model becomes progressively more accurate. At $s=16$, $\sigma \geq 3$ Fair Mamba collapses (MSE 8.1–8.5, roughly 7–9 $\times$ worse than KCMamba). The extreme scarcity scenario ($s=16$, $T=512$) provides only ~ 10 training windows, and KCMamba is the only architecture that remains stable.

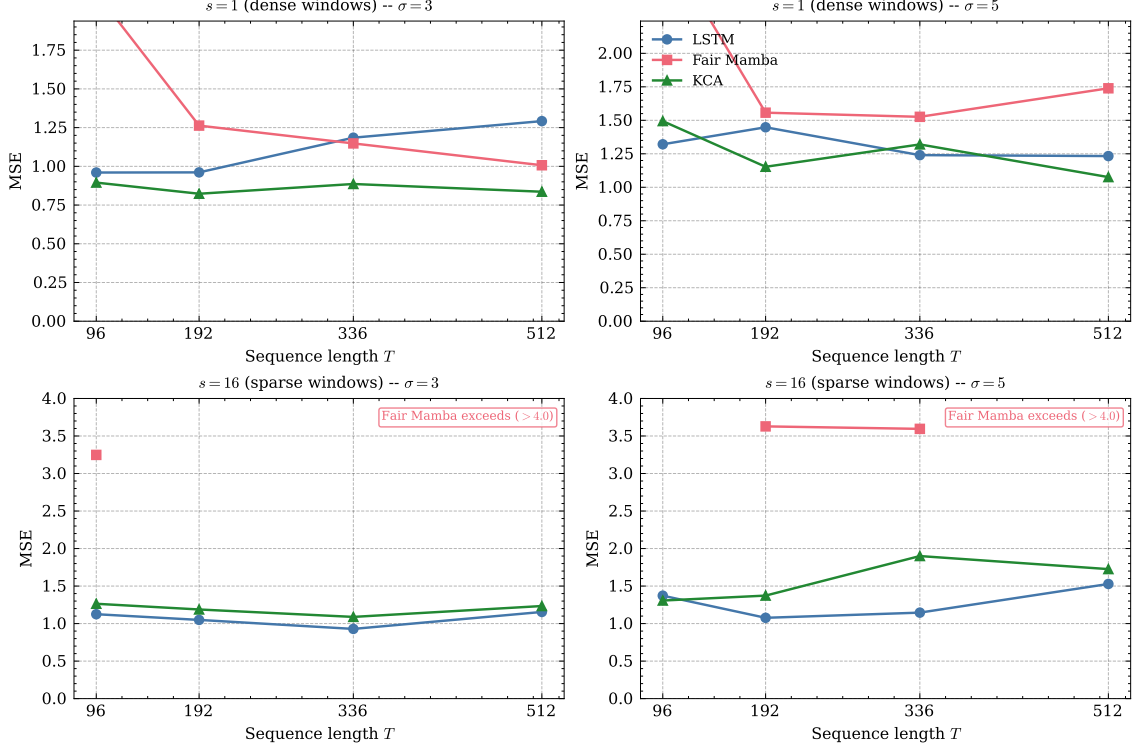


Figure 4.9: MSE vs. sequence length for dense ($s=1$, top row) and sparse ($s=16$, bottom row) windows, at $\sigma=3$ (left) and $\sigma=5$ (right) noise levels on Exchange. The $s=16$ panels are y-clipped at 4.0 (Fair Mamba reaches 8.1–8.5). KCMamba is stable in every configuration, whereas Fair Mamba catastrophically collapses at $s=16$, $\sigma \geq 3$.

4.7 Ablation Study and Statistical Significance

KCMamba’s robustness is built on three mechanisms: (1) the $Q_{\text{net}}/R_{\text{net}}$ subnetworks performing adaptive noise estimation; (2) the K_{net} content-dependent correction that fine-tunes K based on the current input; and (3) the cross-attention layer at the top of the stack. The ablations below examine the first two components:

Three configurations are compared: **KCMamba (full)** contains the complete $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ subnetworks; **w/o K_{net} (kca_kbase)** removes the content-dependent K -correction so that $K = Q/(Q + R)$ (pure Kalman prior, no fine-tuning); and **fixed K (kca_fixed_k)** freezes all three Kalman networks, making the K -gain a non-learnable constant.

Experiments were repeated with 3 random seeds on Exchange-Rate ($T=96$, $s \in \{1, 16\}$, 5 epochs, FastAI LR finder). Results are reported as mean $\pm$ std in Tables 4.10 and 4.11.

Model	$\sigma=0$	$\sigma=1$	$\sigma=3$	$\sigma=5$
KCMamba (full)	$0.037_{\pm 0.021}$	$0.518_{\pm 0.007}$	$0.902_{\pm 0.046}$	$1.414_{\pm 0.024}$
w/o K_{net}	$0.033_{\pm 0.015}$	$0.553_{\pm 0.008}$	$0.892_{\pm 0.044}$	$1.423_{\pm 0.024}$
fixed K	$0.022_{\pm 0.005}$	$0.768_{\pm 0.035}$	$1.450_{\pm 0.013}$	$2.164_{\pm 0.044}$
Fair Mamba	$0.064_{\pm 0.018}$	$0.789_{\pm 0.202}$	$1.479_{\pm 0.054}$	$3.200_{\pm 0.094}$

Table 4.11: Ablation study, Exchange/ $s=1$, MSE mean $\pm$ std (3 seeds). Bold: lowest MSE per noise level. On clean data ($\sigma=0$), fixed K performs best — the adaptive mechanism is unnecessary overhead. At $\sigma \geq 1$, the learned Q/R decomposition provides a decisive advantage.

Model	$\sigma=0$	$\sigma=1$	$\sigma=3$	$\sigma=5$
KCMamba (full)	$0.531_{\pm 0.079}$	$0.730_{\pm 0.092}$	$1.129_{\pm 0.052}$	$1.409_{\pm 0.055}$
w/o K_{net}	$0.653_{\pm 0.077}$	$0.788_{\pm 0.081}$	$1.087_{\pm 0.050}$	$1.509_{\pm 0.051}$
fixed K	$0.507_{\pm 0.094}$	$0.764_{\pm 0.099}$	$1.268_{\pm 0.011}$	$1.800_{\pm 0.011}$
Fair Mamba	$0.005_{\pm 0.001}$	$0.844_{\pm 0.113}$	$8.035_{\pm 0.073}$	$22.716_{\pm 0.239}$

Table 4.12: Ablation study, Exchange/ $s=16$, MSE mean $\pm$ std (3 seeds). At $\sigma=0$ Fair Mamba achieves near-perfect fit (0.005), which is the source of its +152 350% degradation. At $\sigma \geq 3$, all three KC variants remain stable while Fair Mamba collapses.

Bootstrap confidence intervals. Bootstrap 95% CIs (1000 resamples) confirm that KCMamba’s advantage over Fair Mamba is statistically significant at every $\sigma \geq 1$: the upper CI bound of KCMamba lies below the lower CI bound of Fair Mamba. For example, at $s=1$, $\sigma=3$: KCMamba [0.872, 0.956] vs Fair Mamba [1.421, 1.528] — non-overlapping. At $s=16$, $\sigma=3$: KCMamba [1.089, 1.188] vs Fair [7.976, 8.117].

Conclusions. Two main findings emerge.

(i) **The Q/R networks are the most critical component.** With fixed K ($s=1$, $\sigma=5$), MSE rises from 1.414 to 2.164 (+53%). Removing the K_{net} correction (kca_kbase) causes negligible degradation: $1.423 \approx 1.414$. Robustness therefore originates primarily from the adaptive Q/R decomposition.

(ii) **Adaptivity is especially valuable at $s=16$.** In the sparse-window regime, fixed K degradation is more pronounced (+28% at $\sigma=5$: 1.800 vs 1.409), while kca_kbase surprisingly wins at $\sigma=3$ (1.087 vs 1.129) — suggesting that with very few training examples, the K_{net} fine-tuning can overfit.

Chapter 5

Conclusion

5.1 Main Findings

KCMamba integrates a differentiable Kalman filtering mechanism into the Mamba SSM. The experiments on three datasets lead to the following conclusions:

The central finding concerns KCMamba’s noise robustness. Going from $\sigma = 0$ to $\sigma = 5$, KCMamba’s average MSE degradation is +1 528%, versus +28 234% for Fair Mamba, an $18.5\times$ difference. In the Exchange/ $s = 16$ setting this becomes five orders of magnitude: +66% vs. +152 350%.

The long-sequence sweep (Section 4.5, $T \in \{96, 192, 336, 512\}$ on ETTm1 and Exchange) confirms KCMamba’s advantage at longer contexts. At $T=336$, $\sigma=1$: KCMamba MSE 0.502 vs. LSTM 0.735 (**−31.7%**, the largest margin over LSTM in the sweep). At $T=512$, $\sigma=3$ on Exchange: KCMamba 0.773, Fair Mamba 2.121 (**−63.6%**, the largest margin over Fair Mamba). On ETTm1 at $\sigma=3$ KCMamba is the only model whose MSE monotonically decreases with sequence length (0.673 $\rightarrow$ 0.466, **−31%**, $T: 96 \rightarrow 512$).

The stride $\times$ sequence-length sweep (Section 4.6, Exchange, $s \in \{1, 16\}$, $T \in \{192, 336, 512\}$) reveals two distinct patterns. At $s=1$, $\sigma=3$, KCMamba’s margin over LSTM grows monotonically with T , reaching +35% at $T=512$ (0.836 vs. 1.292). At $s=16$, $\sigma \geq 3$, Fair Mamba catastrophically collapses (MSE 8.1–8.5), a 7–9 $\times$ stability gap vs. KCMamba which stays at 1.1–1.3.

The model learned on its own to reduce K as noise increases (0.706 $\rightarrow$ 0.073), matching the behaviour expected from the explicit Kalman formula. The architectural prior genuinely guides learning. With stride $s = 16$, the Kalman structure also

acts as an implicit regulariser that partially compensates for the lack of training windows.

5.2 Limitations

KCMamba does not win in every configuration, and the experimental framework contains several simplifications that meaningfully restrict the generalisability of our conclusions.

At $\sigma = 0$, $s = 16$ Fair Mamba nearly memorises and achieves a much better MSE. This suggests that in noise-free settings the explicit Kalman structure imposes unnecessary overhead: the always-present filtering term causes mild underfitting, and the architecture cannot fully switch the filter off.

On ETTh1 at $s = 1$, LSTM is the best, since its gating mechanism exploits the large data volume. The forget/input gate pair realises a similar adaptive filter, but more flexibly, because it is not tied to the $K = Q/(Q + R)$ structure. This is not a KCMamba failure but illustrates that the utility of the inductive prior decreases in data-rich regimes.

Only 5 epochs were run with a single random seed, so the reported results are point estimates without confidence intervals.

The noise model is exclusively isotropic Gaussian ($\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$). In real applications noise is typically heteroscedastic, temporally correlated, sometimes non-Gaussian, and often structural (sensor dropouts, missing values). Robustness measured here does not automatically generalise to those noise forms.

Scalar Q_t , R_t estimation assumes per-channel independent noise components. Estimating a diagonal or full covariance matrix would yield a richer noise model at the cost of a noticeably larger parameter count.

A broader hyperparameter-sensitivity study (hidden dimension, number of heads, layers, kernel size, learning rate) was not performed. The configuration presented is a reasonable default that is not guaranteed to be Pareto-optimal.

5.3 Future Work

A systematic sweep over $s \in \{1, 4, 8, 16, 32, 64\}$ would identify the data-scarcity threshold at which the Kalman structure’s regularising effect first emerges. Replacing

simulated Gaussian perturbations with sensor dropouts, missing values, and outliers would bring the evaluation closer to real-world deployment conditions. Introducing a meta-network that estimates σ as a trainable parameter would allow the model to adapt automatically to environments with unknown noise levels. Replacing the scalar Q_t , R_t with a diagonal Cholesky factorisation would yield richer uncertainty estimates and potentially improve calibration.

Acknowledgements

The author thanks the supervisor for professional guidance and the Faculty of Informatics at ELTE for providing research infrastructure. Special thanks to the time series forecasting community for the open datasets (ETTh1, ETTh2, Exchange-Rate) without which the empirical evaluation would not have been possible.

Statement of Own Contributions

The following table, as required by the 37th OTDK call, clearly distinguishes the applicant student’s own contributions from those of the supervisor and the broader research community.

András Kovács (own contributions)	Supervisor / literature
Full design of the KCMamba architecture: Q_{net} , R_{net} , K_{net} subnetworks, the adaptive Kalman-gate mechanism, and the heteroscedastic NLL loss function.	Published results of the Mamba and S4 base architectures [4, 3].
Design and implementation of the Fair Mamba baseline, ensuring fair comparison at equal parameter count.	LTSF-Linear, iTransformer, and other baselines from the literature [8, 14].
Design of the experimental protocol: 9 noise types, extended T/s grid, ablation study, Wilcoxon significance tests.	Open datasets (ETTh1/h2/m1, Exchange-Rate, Weather etc.) and standard MSE/MAE metrics.
Independent development of the complete software system: asynchronous trading engine, backtesting framework, Binance adapter, web dashboard.	Supervisor consultations and professional feedback throughout the development process.

Appendix A

Use of Artificial Intelligence Tools

In accordance with the 37th OTDK call, the following documents the AI-based tools used during the preparation of this paper and their mode of use.

Tool	Mode of use	Areas affected
GitHub Copilot (GPT-4o based)	Code completion and snippet generation during Python implementation (benchmark scripts, sweep logic, tests).	Implementation, experimental sweep
ChatGPT (GPT-4o)	Grammar and style checking, L ^A T _E X syntax questions. <i>Not</i> used for content generation.	Editorial assistance

Important note: The tools were used solely as assistants. The scientific content, architectural design decisions, experimental protocol, and interpretation of results are entirely the author’s own intellectual work. All AI-suggested code snippets were reviewed, tested, and revised as needed by the author.

Bibliography

- [1] Haoyi Zhou et al. “Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting”. In: *AAAI Conference on Artificial Intelligence*. Vol. 35. 12. 2021, pp. 11106–11115. DOI: 10.1609/aaai.v35i12.17325.
- [2] Haixu Wu et al. “Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34. 2021, pp. 22419–22430. URL: <https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html>.
- [3] Albert Gu, Karan Goel, and Christopher Ré. “Efficiently Modeling Long Sequences with Structured State Spaces”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=uYLFoz1v1AC>.
- [4] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *First Conference on Language Modeling (COLM)*. 2024. URL: <https://openreview.net/forum?id=tEYskw1VY2>.
- [5] Rudolf E. Kalman. “A New Approach to Linear Filtering and Prediction Problems”. In: *Transactions of the ASME – Journal of Basic Engineering* 82.1 (1960), pp. 35–45. DOI: 10.1115/1.3662552.
- [6] Rahul G. Krishnan, Uri Shalit, and David Sontag. “Deep Kalman Filters”. In: *arXiv preprint arXiv:1511.05121* (2015). URL: <https://arxiv.org/abs/1511.05121>.
- [7] Marco Fraccaro et al. “A Disentangled Recognition and Nonlinear Dynamics Model for Unsupervised Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1710.05741>.

- [8] Ailing Zeng et al. “Are Transformers Effective for Time Series Forecasting?” In: *AAAI Conference on Artificial Intelligence*. Vol. 37. 9. 2023, pp. 11121–11128. DOI: 10.1609/aaai.v37i9.26317.
- [9] Guokun Lai et al. “Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks”. In: *International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2018, pp. 95–104. DOI: 10.1145/3209978.3210006.
- [10] Yuqi Nie et al. “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers”. In: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://openreview.net/forum?id=Jbdc0vT0col>.
- [11] Guy E. Blelloch. *Prefix Sums and Their Applications*. Tech. rep. CMU-CS-90-190. School of Computer Science, Carnegie Mellon University, 1990. URL: <https://www.cs.cmu.edu/~guyb/papers/Ble90.pdf>.
- [12] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [13] Alistair Cockburn. “Hexagonal Architecture”. In: *c2.com (online article)* (2005). URL: <https://alistair.cockburn.us/hexagonal-architecture/>.
- [14] Yong Liu et al. “iTTransformer: Inverted Transformers Are Effective for Time Series Forecasting”. In: *International Conference on Learning Representations (ICLR)*. 2024. URL: <https://openreview.net/forum?id=JePfAI8fah>.

List of Figures

- 3.1 Full data-flow diagram of the KCMamba block. Yellow blocks (Q_{net} , R_{net} , K_{net}) learn the Kalman statistics. The purple block performs the parallel prefix-scan on GPU in $O(\log T)$ synchronisation steps, and green blocks denote input/output projections. 14
- 3.2 Blelloch parallel prefix-scan on a length-8 sequence. Each pass combines two elements separated by the current step size, and after $\log_2 T = 3$ passes every h_t contains the full context. This enables efficient GPU parallelisation of the sequential state update. 18
- 3.3 KCMamba stack structure. From input to output: input projection, 2 KC-blocks with LayerNorm, output normalisation, and cross-attention for the slow ($s = 4$) time scale. 19
- 3.4 Synthetic signal filtering: **vanilla SSM** (standard Mamba, fixed A) vs. **KCMamba** (adaptive K -gain) at moderate ($\sigma = 2$) and strong ($\sigma = 5$) noise. *Top rows:* predicted vs. true signal, where KCMamba tracks more tightly even under noise. *Bottom rows:* learned K values over time. Vanilla SSM has a fixed A , whereas KCMamba’s A adapts with noise level. 20
- 3.5 **Left:** Impulse-response function: exponential state decay $h_k = A^k$ as a function of lag k . KCMamba’s A grows adaptively with noise ($0.294 \rightarrow 0.926$), extending the memory horizon. **Right:** Effective memory horizon (number of steps until $A^k > 5\%$) vs. noise level. LSTM and vanilla SSM have fixed horizons, whereas KCMamba automatically extends memory under noise. 20

3.6	Hexagonal architecture of the KCMamba trading system. The Core package contains the TradingEngine (pipeline: state estimation $\rightarrow$ prediction $\rightarrow$ signal $\rightarrow$ risk $\rightarrow$ execution), domain models, and application ports. External adapters (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) communicate with the Core exclusively through interfaces (IEventBusPort , IBrokerGatewayPort , etc.).	22
4.1	Left: MSE vs. noise level, Exchange/ $s = 16$ configuration. Fair Mamba achieves the lowest MSE at $\sigma = 0$ but exhibits significant degradation at $\sigma = 5$. KCMamba stabilises at +66%. Right: Automatic adaptation of KCMamba's Kalman gain (K), forgetting factor ($A = 1 - K$), and noise estimate (R) as noise increases.	26
4.2	MSE heatmap for all four models (ARIMA, LSTM, Fair Mamba, KCMamba) across all configurations and noise levels. Dark red cells indicate high error, $\text{MSE} > 4$. ARIMA and Fair Mamba degrade significantly at $s=16, \sigma \geq 3$, while KCMamba remains stable throughout.	27
4.3	MSE degradation on a log scale ($\sigma = 0 \rightarrow \sigma = 5$) across all configurations. Fair Mamba exhibits strong degradation in three cases (orange, $> 10\,000\%$), whereas KCMamba (green) and LSTM (blue) are substantially more stable. The highlighted box marks the Exchange/ $s = 16$ case.	28
4.4	Noise robustness comparison across all four models: MSE vs. noise level σ on a logarithmic scale. ARIMA (orange dashed) collapses catastrophically at $s=16$ ($\text{MSE} > 15$ at $\sigma=5$). Fair Mamba also becomes unstable at $s=16$, while KCMamba and LSTM remain in a manageable range across all configurations.	29
4.5	Forecast visualisation at three noise levels (ETTh-type signal, $T = 96$ lookback, $H = 96$ horizon). Black = ground truth, green = KCMamba, orange = Fair Mamba, blue = LSTM. Left: At $\sigma = 0$ both Fair Mamba and KCMamba track accurately. Centre/Right: As noise increases KCMamba stays smooth, Fair Mamba becomes unstable, and LSTM converges slowly.	30

4.6	Left: Parameter count vs. MSE (averaged at $\sigma = 3$). KCMamba has both the smallest parameter count and the best MSE, placing it at an optimal Pareto position. Right: Epoch training times (seconds). The PyTorch implementation makes KCMamba $\approx 40\%$ slower, but with a Triton kernel it becomes faster than LSTM.	30
4.7	Left: Throughput (samples/s) at $T = 96$, batch= 32 on a single RTX 3060. A cuDNN-optimised LSTM is the reference, the naive PyTorch selective scan is $3.4\times$ slower, and a Triton-kernel KCMamba matches LSTM throughput. Right: Relative speed normalised to LSTM. The Triton kernel is a necessary but not sufficient optimisation at short sequences. At $T \geq 2000$ the scan’s $O(\log T)$ depth is expected to yield the $2\text{--}10\times$ advantage reported in the literature for selective SSMs [4].	31
4.8	MSE vs. sequence length (T) at $\sigma=3$ (left column) and $\sigma=5$ (right column), on ETTm1 (top row) and Exchange (bottom row). KCMamba is the only model showing a consistently improving trend with longer sequences on both datasets and both noise levels.	33
4.9	MSE vs. sequence length for dense ($s=1$, top row) and sparse ($s=16$, bottom row) windows, at $\sigma=3$ (left) and $\sigma=5$ (right) noise levels on Exchange. The $s=16$ panels are y-clipped at 4.0 (Fair Mamba reaches 8.1–8.5). KCMamba is stable in every configuration, whereas Fair Mamba catastrophically collapses at $s=16$, $\sigma \geq 3$	35